Lazy Loading

React.lazy is used to load a component's JavaScript file only when it is actually needed, instead of loading it in the main bundle at the start. This helps reduce the initial bundle size and makes the app load faster on first visit. It is about loading component code on demand — not about fetching API data. When the component file is being downloaded, Suspense can temporarily show a fallback UI (like a spinner).

Modern React apps (especially MERN apps like you're building) grow large

👉 Result:

- **Large initial bundle**
- **Slow first load**
- **Worse Core Web Vitals**
- **Poor mobile performance**

This is called eager loading.

We need:

Load code only when needed.

This is called code splitting + lazy loading.

Code Splitting (Foundation Concept)

What is Code Splitting?

Breaking your JS bundle into multiple smaller files (chunks) that load on demand.

Bundlers like:

- **Webpack**
- **Vite**
- **Parcel**

What is React.lazy?

React.lazy allows you to render a dynamic import as a component.

Syntax

```
const Dashboard = React.lazy(() => import("./Dashboard"));
```

Important:

- **Must return a default export**
- **Must wrap with <Suspense>**

🔽 What is Suspense?

Suspense is a React component that:

Shows a fallback UI while something is loading.

Originally built for:

- **Code splitting**
- **Later extended for data fetching**

Step 1: Normal Component

```
// Dashboard.js
export default function Dashboard() {
  return <h1>Dashboard Loaded</h1>;
}
```

Step 2: Lazy Import

```
import React, { Suspense } from "react";

const Dashboard = React.lazy(() => import("./Dashboard"));

function App() {
return (
<div>
<Suspense fallback={<div>Loading...</div>}>
<Dashboard />
</Suspense>
</div>
);
}

export default App;
```

What Happens Internally (Interview Gold)

When React sees:

```
const Dashboard = React.lazy(() => import("./Dashboard"));
```

Internally:

1. **React calls the function**
2. **It returns a Promise**
3. **React throws that Promise**
4. **Suspense catches it**
5. **Suspense shows fallback**
6. **When Promise resolves:**
    - **React re-renders**
    - **Component appears**

This is called "throwing a Promise for Suspense".

Lazy Loading with React Router (Real-World Usage)

This is the most common pattern in production apps.

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import React, { Suspense } from "react";

const Home = React.lazy(() => import("./pages/Home"));
const Dashboard = React.lazy(() => import("./pages/Dashboard"));

function App() {
  return (
    <BrowserRouter>
      <Suspense fallback={<div>Loading Page...</div>}>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/dashboard" element={<Dashboard />} />
        </Routes>
      </Suspense>
    </BrowserRouter>
  );
}
```

Now:

- **/ loads Home chunk**
- **/dashboard loads Dashboard chunk only when visited**

Huge performance win.

Advanced Suspense Patterns

1 Nested Suspense

```
<Suspense fallback={<PageLoader />}>
  <Dashboard>
    <Suspense fallback={<ChartLoader />}>
      <Chart />
    </Suspense>
```

```
  </Dashboard>
</Suspense>
```

Allows:

- **Progressive loadin**
- **Better UX**

2️ Error Boundaries (VERY IMPORTANT)

If lazy import fails (network error), Suspense doesn't handle errors.

You need an Error Boundary.

```
class ErrorBoundary extends React.Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  render() {
    if (this.state.hasError) {
      return <h1>Something went wrong.</h1>;
    }

    return this.props.children;
  }
}
```

```
<ErrorBoundary>
<Suspense fallback={<div>Loading...</div>}>
<Dashboard />
</Suspense>
</ErrorBoundary>
```

When NOT to Use React.lazy

Avoid lazy loading:

- **Small components**
- **Frequently used components**
- **Above-the-fold UI**
- **Components used immediately**

Lazy loading adds:

- **Extra network request**
- **Slight delay on first load**

So don’t over-optimize.

Performance Deep Dive

Without Lazy:

main.js (800kb)

With Lazy:

main.js (200kb)
dashboard.chunk.js (300kb)
admin.chunk.js (300kb)

First load faster.

Difference: Lazy Loading vs Tree Shaking

Lazy Loading	Tree Shaking
Runtime loading	Build-time removal
Splits bundles	Removes unused code
Uses dynamic import	Uses ES modules

Q.Why must Suspense wrap lazy components?

Because React.lazy returns a Promise and Suspense handles the loading state.

Advanced: Preloading Strategy

Sometimes you want:

- **Lazy load**
- **But preload when user hovers**

Example:

```
const Dashboard = React.lazy(() => import("./Dashboard"));

const preloadDashboard = () => import("./Dashboard");
```

Use:

```
<button onMouseEnter={preloadDashboard}>
  Go to Dashboard
</button>
```

Improves UX massively.

Data Loading

Lazy loading solves code loading.
But your question is about data loading.

These are two different problems:

Problem	Tool
Load component JS	React.lazy
Load data (API call)	useEffect, React Query, Suspense for data

React.lazy handles component-level code splitting. For dynamic data, we use useEffect or a data-fetching library like React Query. Suspense can coordinate both code and data loading for cleaner async UI.

1️⃣ Traditional Way (Most Common)

Use useEffect + useState

```
import { useEffect, useState } from "react";

function Users() {
  const [users, setUsers] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("/api/users")
      .then(res => res.json())
```

```
      .then(data => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <div>Loading...</div>;

  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

What happens:

1. Component renders
2. useEffect runs
3. Fetch starts
4. Re-render when data arrives

This is client-side data fetching.

2️⃣ Problem With This Approach

- Manual loading state
- Manual error state

- Multiple re-renders
- No caching
- No background refetch

For senior-level apps → use React Query (TanStack Query).

3️⃣ Production Way (React Query)

npm install @tanstack/react-query

```
import { useQuery } from "@tanstack/react-query";

function Users() {
  const { data, isLoading, error } = useQuery({
    queryKey: ["users"],
    queryFn: () =>
      fetch("/api/users").then(res => res.json()),
  });

  if (isLoading) return <div>Loading...</div>;
  if (error) return <div>Error!</div>;

  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
```

```
  );
}
```

Benefits:

- Caching
- Background refetch
- Deduplication
- Smart retries
- Clean API

This is how scalable apps do it.

4️⃣ Suspense for Data (Advanced React 18)

1️⃣ What Normally Happens (Without Suspense)

```
function Users() {
  const { data, isLoading } = useQuery(...);

  if (isLoading) return <Spinner />;

  return <UserList data={data} />;
}
```

Here:

- **Component renders**
- **It checks isLoading**
- **You manually return loading UI**

You control loading.

2️⃣ What Happens With Suspense

```
<Suspense fallback={<Spinner />}>
  <Users />
</Suspense>
```

And inside:

```
const { data } = useQuery({
  queryKey: ["users"],
  queryFn: fetchUsers,
  suspense: true
});
```

Now:

- **If data is NOT ready**
- **React Query throws a Promise**
- **React catches it**
- **Suspense shows fallback**
- **When Promise resolves → React renders <Users />**

You no longer check isLoading.

React controls async rendering.